

Aula 11 – Técnicas de Programação SQL

Banco de Dados – Guia de Revisão

Contents

1 Objetivo da aula

Esta aula mostra como programas acessam bancos de dados. O foco é entender a diferença entre SQL interativo e SQL usado dentro de aplicações, além de JDBC, SQL embutido, cursores, Hibernate, procedures e SQL Injection.

2 Ideia central

Na prática, usuários raramente digitam SQL diretamente. Aplicações executam SQL por meio de uma linguagem hospedeira, como Java, C, C++ ou Python.

3 Linguagem hospedeira e sublinguagem de dados

- **Linguagem hospedeira:** linguagem geral da aplicação, como Java.
- **Sublinguagem de dados:** SQL usado para acessar o banco.

4 Descompasso de impedância

Ocorre porque linguagens de programação e bancos relacionais representam dados de formas diferentes.

Exemplo:

- Java usa objetos, listas, tipos, referências.
- SQL retorna tabelas, linhas e colunas.

Por isso são necessários cursores, iteradores, `ResultSet` ou frameworks ORM.

5 SQL embutido

SQL embutido coloca comandos SQL dentro de uma linguagem hospedeira. Em C, por exemplo, comandos podem aparecer com `EXEC SQL`.

Conceitos importantes:

- Variáveis compartilhadas entre programa e SQL.
- Pré-processador para separar comandos SQL.
- `SQLCODE` ou `SQLSTATE` para erros.
- Cursor para resultados com várias linhas.

6 Cursores

Cursores permitem processar resultado linha por linha.

Fluxo típico:

```
DECLARE cursor  
OPEN cursor  
FETCH cursor  
CLOSE cursor
```

FETCH avança para a próxima tupla.

7 SQL dinâmica

SQL dinâmica permite montar comandos em tempo de execução. É útil quando a consulta depende de escolhas do usuário, mas aumenta riscos se for feita com concatenação insegura.

8 JDBC

JDBC é uma API Java para acesso a banco de dados.

Fluxo comum:

1. Abrir `Connection`.
2. Criar `Statement` ou `PreparedStatement`.
3. Executar comando.
4. Processar `ResultSet`, se houver.
5. Fechar recursos.

8.1 Exemplo visual: consulta segura

```
Connection con = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM Aluno WHERE matricula = ?"
);
ps.setInt(1, matricula);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    String nome = rs.getString("nome");
}
```

- `Connection`: conexão com o banco.
- `PreparedStatement`: comando parametrizado.
- `?`: parâmetro.
- `setInt(1, ...)`: preenche o primeiro parâmetro.
- `ResultSet`: linhas retornadas.

9 executeQuery e executeUpdate

- `executeQuery()`: usado para `SELECT`; retorna `ResultSet`.
- `executeUpdate()`: usado para `INSERT`, `UPDATE`, `DELETE`; retorna quantidade de linhas afetadas.

10 Transações em JDBC

Para controlar transação manualmente:

```
con.setAutoCommit(false);
try {
    // comandos SQL
    con.commit();
} catch (Exception e) {
    con.rollback();
}
```

11 SQL Injection

SQL Injection ocorre quando entrada externa é concatenada diretamente no SQL.

Código vulnerável:

```
String sql = "SELECT * FROM Usuario WHERE login='" + login +
            "' AND senha='" + senha + "'";
```

Entrada perigosa:

```
' OR '1'='1
```

Correção: usar `PreparedStatement` com parâmetros.

12 Hibernate e ORM

Hibernate é um framework de mapeamento objeto-relacional. Ele associa classes Java a tabelas relacionais.

Exemplo conceitual:

Java	Banco Relacional
Classe Aluno	Tabela Aluno
Objeto	Linha
Atributo do objeto	Coluna

13 Procedures, funções e SQL/PSM

Procedures e funções armazenadas ficam no SGBD e podem ser chamadas por aplicações.

Vantagens:

- centralizam lógica perto dos dados;
- reduzem repetição de código;
- podem melhorar controle e padronização.

14 Senhas e dados sensíveis

Senhas não devem ser armazenadas em texto puro. Prática melhor: hash com salt.

Fluxo:

1. Usuário digita senha.
2. Sistema combina senha com salt.
3. Calcula hash.
4. Compara com hash armazenado.

15 Pegadinhas comuns

- `PreparedStatement` não serve só para performance; também ajuda contra SQL Injection.
- `ResultSet` começa antes da primeira linha; precisa de `rs.next()`.
- `executeQuery` é para consulta; `executeUpdate` é para alteração.
- Hibernate não elimina o modelo relacional; ele mapeia objetos para relações.

16 Desafios

Desafio 1

Explique por que o código abaixo é vulnerável:

```
String sql = "DELETE FROM Employee WHERE ssn=" + ssn;
```

Desafio 2

Qual método JDBC você usaria para executar um `SELECT`? E para executar um `UPDATE`?

17 Resolução dos desafios

Desafio 1: concatena entrada diretamente no SQL. Um valor malicioso pode alterar a condição e afetar mais linhas que o esperado.

Desafio 2: `executeQuery()` para `SELECT`; `executeUpdate()` para `UPDATE`, `INSERT` e `DELETE`.

18 Checklist final

- Sei explicar descompasso de impedância?
- Sei a função de `Connection`, `PreparedStatement` e `ResultSet`?
- Sei reconhecer SQL Injection?
- Sei diferenciar cursor, JDBC, SQLJ, Hibernate e stored procedure?